

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 3

Disciplina: Tehnici de programare

Tema: ”Însușirea Python”

A efectuat :

st. gr.SI – 261 Ovăs Viorel

A verificat :

asistent universitar Emil Cebotaros

Chișinău – 2026

SARCINA LUCRĂRII

script.py

```
1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8     print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

IPython Shell

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]:

Great job!

Solution

Submit

script.py

```
1 numbers = []
2 strings = []
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 numbers += [1, 2, 3]
7 strings += ["hello", "world"]
8 second_name = names[1]
9
10 # this code should write out the filled arrays and
11 # the second name in the names list (Eric).
12 print(numbers)
13 print(strings)
14 print("The second name on the names list is %s" %
15       second_name)
```

IPython Shell

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is Eric

In [1]:

Great Job!

Solution

Submit

script.py

```
2 y = object()
3
4 # TODO: change this code
5 x_list = [x] * 10
6 y_list = [y] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len
    (big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) == 10
    :
15     print("Almost there...")
```

IPython Shell

<script.py> output:

x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
Almost there...
Great!

In [1]:

Good work!

Solution

Submit

script.py

```
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance
    is $%.s."
3
4 print(format_string % data)
```

IPython Shell

<script.py> output:

Hello John Doe. Your current balance is \$53.44.

In [1]:

Great work!

Solution

Submit

script.py

```

2
3 # Length should be 20
4 print("Length of s = %d" % len(s))
5
6 # First occurrence of "a" should be at index 8
7 print("The first occurrence of the letter a = %d"
8     % s.index("a"))
9
10 # Number of a's should be 2
11 print("a occurs %d times" % s.count("a"))
12
13 # Slicing the string into bits
14 print("The first five characters are '%s'" % s[:5])
15 ) # Start to 5
16 print("The next five characters are '%s'" % s[5:10]) # 5 to 10

```

IPython Shell

```

String in uppercase: HEY THERE! WHAT SHOULD THIS STRING
BE?
String in lowercase: hey there! what should this string
be?
Split the words of the string: ['Hey', 'there!', 'what'
, 'should', 'this', 'string', 'be?']

<script.py> output:
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs an'
The thirteenth character is 'a'

```

Great work!

Solution

Submit

Exercise

Change the variables in the first section, so that each if statement resolves as True.

script.py

```

1 # change this code
2 number = 16
3 second_number = 0
4 first_array = [1,2,3]
5 second_array = [1,2]
6
7 * if number > 15:
8     print("1")
9
10 * if first_array:
11     print("2")
12
13 * if len(second_array) == 2:
14     print("3")
15
16 * if len(first_array) + len(second_array) == 5:
17     print("4")

```

IPython Shell

```

<script.py> output:
1
2
3
4
5
6

In [1]: |

```

Great Work!

Solution

Submit

that come after 237 in the sequence.

script.py

```
162,
7   758, 219, 918, 237, 412, 566, 826, 248, 866,
950,
8   626, 949, 687, 217, 815, 67, 104, 58, 512, 24,
9   892, 894, 767, 553, 81, 379, 843, 831, 445,
742,
10  717, 958, 609, 842, 451, 688, 753, 854, 685,
93,
11  857, 440, 380, 126, 721, 328, 753, 470, 743,
527
12 ]
13
14 # your code goes here
15 for number in numbers:
16     if number == 237:
17         break
```

IPython Shell

```
592
236
942
386
462
418
344
236
566
978
328
162
758
918
```

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join [over a](#)

3. Run and see all the functions work together!

script.py

```
4     "More organized code",
5     "More readable code",
6     "Easier code reuse",
7     "Allowing programmers to share and connect
code together"
8 ]
9
10 # Modify this function to concatenate to each
benefit - " is a benefit of functions!"
11 def build_sentence(benefit):
12     return benefit + " is a benefit of functions!"
13
14 def name_the_benefits_of_functions():
15     list_of_benefits = list_benefits()
16     for benefit in list_of_benefits:
17         print(build_sentence(benefit))
```

IPython Shell

<script.py> output:

```
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code together
is a benefit of functions!
```

In [1]:

Nice work!

Solution

Submit

script.py

```
1 # define the Vehicle class
2 class Vehicle:
3     name = ""
4     kind = "car"
5     color = ""
6     value = 100.00
7     def description(self):
8         desc_str = "%s is a %s %s worth $%.2f." %
9         (self.name, self.color, self.kind, self.value)
10         return desc_str
11 # your code goes here
12 car1 = Vehicle()
13 car1.name = "Fer"
14 car1.color = "red"
15 car1.kind = "convertible"
16 car1.value = 60000.00
```

IPython Shell

<script.py> output:

```
Fer is a red convertible worth $60000.00.
Jump is a blue van worth $10000.00.
```

In [1]:

Great job!

Solution

Submit

script.py

```
1 phonebook = {
2     "John" : 938477566,
3     "Jack" : 938377264,
4     "Jill" : 947662781
5 }
6
7 # your code goes here
8 phonebook["Jake"] = 938273443
9 del phonebook["Jill"]
10
11 # testing code
12 if "Jake" in phonebook:
13     print("Jake is listed in the phonebook.")
14 if "Jill" not in phonebook:
15     print("Jill is not listed in the phonebook.")
```

IPython Shell

<script.py> output:

```
Jake is listed in the phonebook.
Jill is not listed in the phonebook.
```

In [1]:

Nice work!

Solution

Submit

script.py

```
1 import re
2
3 # Your code goes here
4 find_members = []
5 for member in dir(re):
6     if "find" in member:
7         find_members.append(member)
8
9 print(sorted(find_members))
```

IPython Shell

<script.py> output:

['findall', 'finditer']

In [1]:

Great work!

Solution

Submit

will simultaneously switch the values of a and b.

script.py

```
1 # fill in this function
2 def fib():
3     a, b = 1, 1
4     while True:
5         yield a
6         a, b = b, a + b
7
8 # testing code
9 import types
10 if type(fib()) == types.GeneratorType:
11     print("Good, The fib function is a generator.")
12
13 counter = 0
14 for n in fib():
15     print(n)
16     counter += 1
17     if counter == 10:
18         break
```

IPython Shell

<script.py> output:

Good, The fib function is a generator.

1

1

2

3

5

8

13

21

34

55

Good work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today! [Take Tech Bootcamps](#)

from the list, as integers. [Scripting Languages](#)

script.py

```
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
2 newlist = [int(x) for x in numbers if x > 0]
3 print(newlist)
```

IPython Shell

<script.py> output:
[34, 44, 68, 44, 12]

In [1]:

Very nice!

Solution

Submit



This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1 l = [2, 4, 7, 3, 14, 19]
2 * for i in l:
3     # your code here
4     is_odd = lambda x: x % 2 != 0
5     print(is_odd(i))
```

IPython Shell

<script.py> output:
False
False
True
True
False
True

In [1]:

Nice work!

Solution

Submit



This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today! [Take Tech Bootcamps](#)

otherwise.

script.py

```
1 # edit the functions prototype and implementation
2 def foo(a, b, c, *args):
3     return len(args)
4
5 def bar(a, b, c, **kwargs):
6     return kwargs.get("magicnumber") == 7
7
8 # test code
9 if foo(1, 2, 3, 4) == 1:
10     print("Good.")
11 if foo(1, 2, 3, 4, 5) == 2:
12     print("Better.")
13 if bar(1, 2, 3, magicnumber=6) == False:
14     print("Great.")
15 if bar(1, 2, 3, magicnumber=7) == True:
16     print("Awesome!")
```

IPython Shell

<script.py> output:

```
Good.
Better.
Great.
Awesome!
```

In [1]:

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join over a

script.py

```
1 # Setup
2 actor = {"name": "John Cleese", "rank": "awesome"}
3
4 # Function to modify!!!
5 def get_last_name():
6     try:
7         return actor["last_name"]
8     except KeyError:
9         return actor["name"].split()[1]
10
11 # Test code
12 get_last_name()
13 print("All exceptions caught! Good job!")
14 print("The actor's last name is %s" % get_last_name())
```

IPython Shell

<script.py> output:

```
All exceptions caught! Good job!
The actor's last name is Cleese
```

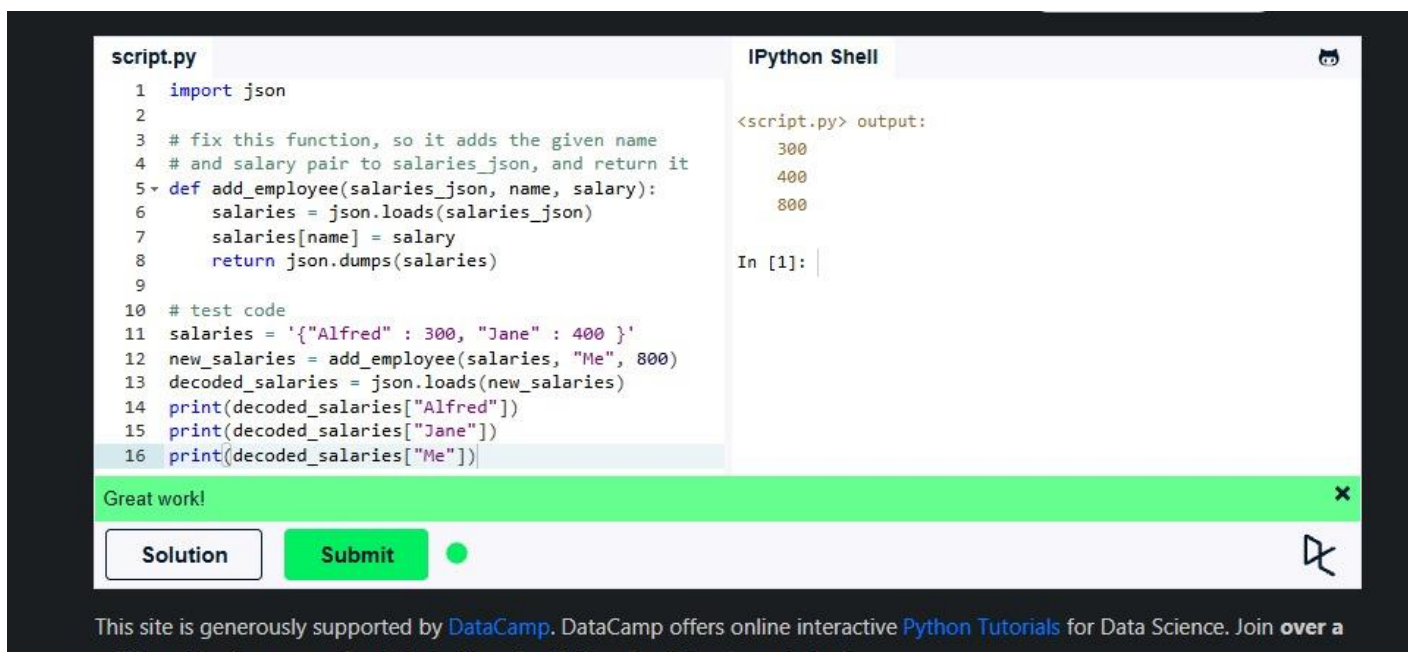
In [1]:

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join over a



The screenshot shows a code editor with a file named `script.py` and an adjacent IPython Shell. The script defines a function `add_employee` that takes a JSON string of salaries, a name, and a salary, and returns a new JSON string with the employee added. It then tests this function with a sample JSON and prints the results. The IPython Shell shows the output of the script: 300, 400, and 800. A green notification bar at the bottom says "Great work!". Below the editor are buttons for "Solution" and "Submit".

```
script.py
1 import json
2
3 # fix this function, so it adds the given name
4 # and salary pair to salaries_json, and return it
5 def add_employee(salaries_json, name, salary):
6     salaries = json.loads(salaries_json)
7     salaries[name] = salary
8     return json.dumps(salaries)
9
10 # test code
11 salaries = '{"Alfred" : 300, "Jane" : 400 }'
12 new_salaries = add_employee(salaries, "Me", 800)
13 decoded_salaries = json.loads(new_salaries)
14 print(decoded_salaries["Alfred"])
15 print(decoded_salaries["Jane"])
16 print(decoded_salaries["Me"])

IPython Shell
<script.py> output:
300
400
800

In [1]:
```

Great work!

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for Data Science today.



Concluzii

În urma realizării acestei lucrări de laborator, am studiat și exersat noțiunile de bază ale limbajului de programare Python. Am învățat să utilizez variabile, tipuri de date, operatori, condiții, cicluri repetitive și funcții pentru realizarea diferitelor programe. Exercițiile practice m-au ajutat să înțeleg mai bine logica programării și modul de rezolvare a problemelor prin intermediul codului. De asemenea, am dobândit mai multă experiență în scrierea și verificarea programelor, precum și în identificarea și corectarea erorilor. Consider că aceste cunoștințe reprezintă o bază importantă pentru aprofundarea programării și pentru studiul ulterior a unor concepte mai avansate în domeniul informaticii.

